NEXUS — Network-aware EXecution and Unified Scheduling: Technical Report

Oct 25, 2026 · @Aswin Mony

2. Host analysis and simulation design

The host can run up to 48 node processes faithfully, so the 12-node cluster runs as 12 real OS processes; task kernels are time-emulated because 2 vCPUs cannot execute 176 emulated cores. `nexus_launcher` profiles the machine before every experiment and refuses a cluster larger than its estimate.

Probe	Measured on this host
CPU	Intel Xeon @ 2.30 GHz, 2 vCPU (KVM guest), AVX-512 / AMX
Memory	8.0 GB total, 7.4 GB available, no swap
Shared memory (/dev/shm)	7.9 GB
Open files / user processes	20,000 / 32,026
Loopback MTU / UDP rmem_max	65,536 B / 4 MB
1 ms sleep overshoot (p50 / p99)	78 µs / 96–131 µs
UDP loopback RTT between processes (p50 / p99)	28 µs / 55 µs
fork + exec + wait	1.25 ms
Atomic CAS	8 ns/op

Sizing rule: node processes are mostly idle (an event loop plus sleeping worker threads), so the binding limits are memory (≈15 MB budgeted per node, half of available RAM) and event-loop CPU (24 nodes per vCPU), capped at 64 by the fabric segment. Timer overshoot of ~0.1 ms means tasks shorter than 0.05 s would carry more than 1% timing error; the shortest workload task is 0.5 s.

What is real and what is emulated

- Real: one process per node, UDP sockets, the wire protocol, every thread (event loop, transmit-delay thread, worker pool), timers, clocks, reservation and fault-handling logic, process kill/freeze/restart.

- Emulated network: a shared-memory fabric (`/dev/shm`) models NICs, rack uplinks and the core switch. Each link is a lock-free reservation timeline updated with CAS by every process, so congestion from one node is felt by all. Datagrams are held until their computed arrival time and stamped with an emulated NIC receive timestamp.
- Emulated hardware: a task occupies a worker thread for `work ÷ (resources × true speed)` × interference × log-normal noise (σ = 0.06). True speeds differ from advertised ones (a2 CPUs run at 0.6×, g1 GPUs at 0.55×) and are never visible to schedulers.

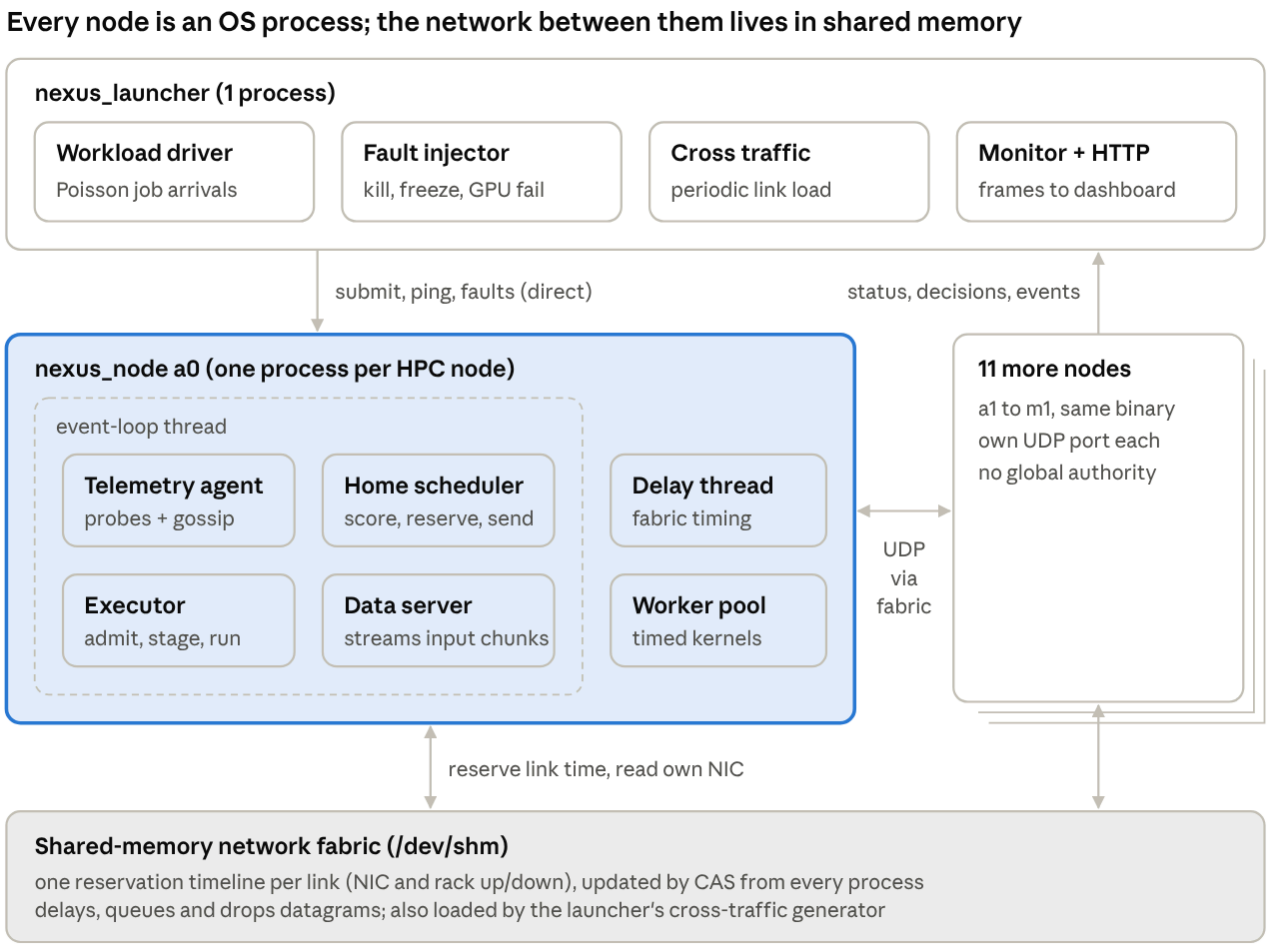
Emulated cluster (12 nodes, 176 cores, 8 GPUs, 3 racks)

Rack	Nodes	Per node	Partition	Notes
A-compute (5 Gb/s uplink)	a0–a3	16 cores, 64 GB, 2.5 Gb/s NIC	general	a1 holds dataset D0; a2 secretly 40% slower; a1 slowed 1.8× from 30–55 s
B-gpu (4 Gb/s uplink)	g0–g3	8 cores, 96 GB, 2 GPUs	gpu	g0 and g2 hold D2; g1 GPUs secretly 0.55×; g3 GPUs 1.5×
C-data (10 Gb/s uplink)	s0, s1	8 cores @0.8, 32 GB, 5 Gb/s NIC	storage	s0 holds D0, D2; s1 holds D1
C-data	m0, m1	32 cores @0.9, 512 GB	bigmem	m0 holds D1; m1 slowed 1.5× from 20–60 s

Rack C sits 1.2 ms from rack A and 0.9 ms from rack B. Three periodic cross-traffic sources (rack B downlink, s0 uplink, rack C uplink, 30–50% duty) create busy and quiet network periods. Tenants enforce access policy and quotas: physics may use general/storage/gpu at ⩽75% of a node, ml may use gpu/general, bio may use general/bigmem/storage at ⩽75%.

3. Architecture

A run is 13 processes: the launcher and one `nexus_node` per HPC node, all from the same 5,900-line C++17 code base with no external dependencies.



process architecture · 13 processes, 1 shared-memory segment

Data-plane datagrams pass through the fabric, which computes each one's arrival time; a per-node delay thread releases it then. Management traffic (submissions, pings, fault commands, monitoring) goes direct, like an out-of-band management network.

Threads per node: one event loop (poll on the UDP socket and an eventfd, plus a timer heap) owns all scheduling state, so it needs no locks; one transmit-delay thread; and a worker pool sized to the node's cores that occupies a slot for each running task and reports completion through the eventfd.

Wire protocol: 24 message types in a little-endian binary encoding with bounds-checked decoding. The main ones: `Probe` / `ProbeAck` (telemetry + gossip), `ReserveReq` / `ReserveRep`, `Delegate` / `DelegateAck`, `TaskStarted` / `TaskDone` / `DoneAck` / `TaskFailed` / `TaskMoved`, `TaskQuery`, `Fetch` / `DataDone`, `StateReq` / `StateRep` (centralized baseline), `Submit` / `SubmitAck`, `Ping` / `Pong`, `Fault`.

Code layout: `src/net` (fabric, transport), `src/core` (cost model), `src/node` (event loop, telemetry, home, executor), `src/launcher` (host analysis, workload, experiment runner, HTTP), `web/dashboard.html`, `tests/`.

4. Telemetry discovery

Each node learns the cluster through probes that also carry gossip, under a token bucket of 64 kB/s per node that covers probes, acknowledgements and state replies alike.

Probe round. Every interval the node probes its 3 peers with the oldest direct measurement. A latency probe carries the sender's own record plus the 4 freshest third-party records it holds (epidemic gossip). The responder answers with its own record and gossip.

RTT from hardware timestamps. The fabric stamps each datagram with its emulated NIC receive time, so one-way latency is estimated as

```
\hat{d} = \tfrac{1}{2}\big[(t^{rx}_{ack} - t^{tx}_{probe}) - (t^{tx}_{ack} - t^{rx}_{probe})\big]
```

which removes the responder's processing time. Samples feed an EWMA with variance ($\alpha = 0.25$).

Packet-pair bandwidth. Every 6th probe to a peer, and only while the local NIC is quiet, the node sends two back-to-back 32 kB probes. The receiver reports the arrival gap; available bandwidth = bytes ÷ gap. Real data transfers are also used as passive bandwidth samples, so busy links are measured for free.

Adaptive interval. Starting from 400 ms, the interval shrinks with latency volatility, halves while any peer is unknown or stale, doubles in a quiet and stable cluster, and stretches 1.5× while the local NIC is above 60% utilisation. Bandwidth probes are suppressed above 30% NIC utilisation; a whole round is deferred (up to the 3 s ceiling) while the NIC is busy. Bounds: 150 ms – 3 s. Two fixed policies (100 ms and 2 s) serve as ablations.

Time-aware state. Every record carries its origin timestamp. A view entry's freshness is exp(−age ÷ 2 s). Entries older than 4 s are *stale* (usable, penalised); older than 6 s are *suspected* (excluded from placement and treated as failed by homes). Link estimates widen with age: the bandwidth lower bound loses 3% per second of age (up to 60%) and the latency deviation grows in proportion.

What a record carries (≈330 B): capabilities (cores, memory, GPUs, advertised speeds, partition, datasets held), free resources, queue length, queued core- and GPU-seconds, health bits, recent failure rate, NIC utilisation, learned per-class correction factors, live per-tenant core usage, and the node's own link estimates to every peer (so third parties can estimate paths they never probe).

5. Scheduling, delegation and fault handling

The node a job is submitted to becomes its *home*: it tracks the job's DAG, scores every node for each ready task, reserves capacity on the winner and delegates the task. No node holds global authority.

Cost model. For task *t* on candidate *n*, seen from the home's view at time *now*:

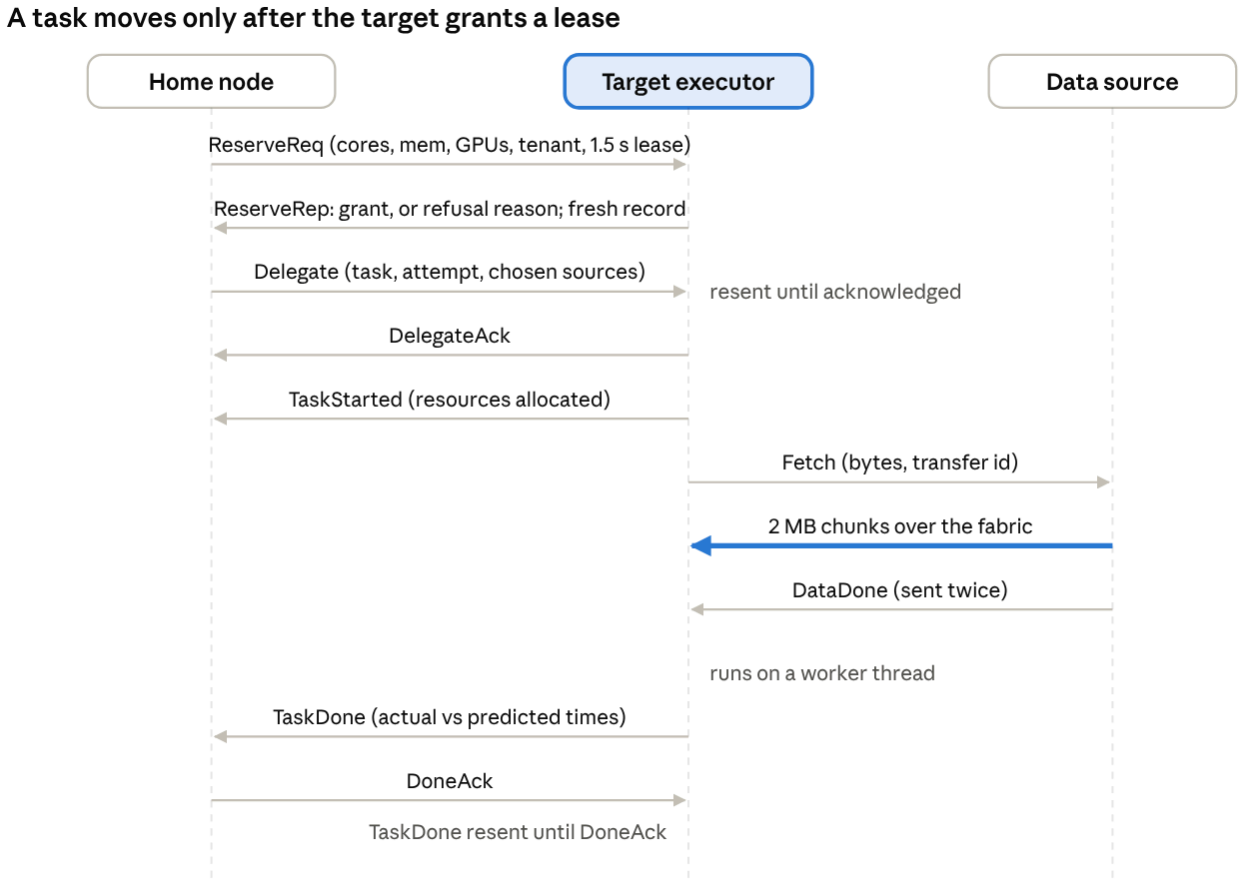
```
C(t,n) = w_q W_n + w_x X_{t,n} + w_e E_{t,n} + w_r\,p_n\,(W_n + X_{t,n} +
E_{t,n}) + w_u U_{t,n}
```

- W (queueing): advertised queued core-seconds ÷ cores, minus the record's age, plus delegations this home sent that the record does not yet show. GPU tasks use the GPU backlog.
- X (staging): for each input, the cheapest live replica or parent-output location: latency + latency s.d. + bytes ÷ bandwidth lower bound, inflated by loss; local reads use disk bandwidth.
- E (execution): work ÷ (cores or GPUs × advertised speed) × the node's learned correction factor for that workload class.
- p (failure probability): recent failure rate + 0.1 if degraded + a staleness ramp from 2 s to 6 s of record age.
- U (uncertainty): half the gap between pessimistic and mean staging time + 5% of (wait + exec) per second of record age.

Class weights encode workload sensitivity: interactive tasks weight queueing and uncertainty 1.5×, data-ETL weights staging 1.25×, GPU training weights risk 1.5× (long and expensive to redo), memory-heavy weights risk 1.2×.

Hard constraints before scoring: static capacity (cores, memory, GPUs), GPU and CPU health, tenant access to the node's partition, tenant quota (static and the live per-tenant usage the node advertises), and liveness (suspected nodes are excluded). The executor re-checks all of them authoritatively when it answers a reservation.

Delegation protocol. The home keeps the task if its own cost is within 10% of the best; otherwise it sends a reservation request with a 1.5 s lease. The target admits it only if capacity, quota, policy, health and a 6-task queue limit allow, and replies with a fresh record either way. On a grant the home sends the task; on a refusal it merges the fresh record and moves to the next candidate (up to 4, then backs off 300 ms). Every step has an identifier (task, epoch, attempt) and is idempotent, so retransmissions are safe: delegations are resent until acknowledged, completions are resent until the home acknowledges them, and data-completion notices are sent twice.



delegation protocol · home, target, data source

The lease is the concurrency control: two homes racing for the same node are serialised by the target, which grants against its true state, so a node is never overbooked by stale views.

Re-delegation. Once a second, an executor re-scores its queued tasks that have waited over 1 s; if another node is at least 35% cheaper it reserves there and hands the task off, keeping it frozen locally until the new owner acknowledges (at most 2 moves per task, one hand-off in flight per node).

Feedback. Each completion compares actual with advertised execution time; the executing node keeps a per-class EWMA of the ratio and advertises it, so every home learns that a2 and g1 are slower than they claim. Actual transfer times update link bandwidth estimates.

Fault handling.

- Executor suspected (record older than 6 s): the home reschedules the task as a new attempt, excluding that node.
- Executor alive but task unknown (lost hand-off or delegation): the home queries the executor every 3 s and reschedules after two denials.

- Input source unreachable: the fetch is re-queried, then switched to another replica, then the task fails back to its home.
- GPU fault: running GPU tasks are cancelled and reported; the node advertises GPU-down and refuses GPU reservations.
- Home node lost: the submitting client notices missing heartbeats after 3 s and re-submits the job's unfinished tasks, with completed outputs, to a new home under a higher epoch. Late duplicate completions are discarded by epoch.

Baselines (same processes, same executor, same constraints):

Policy	Who decides	How
NEXUS	home node of each job	full cost model, reservations, re-delegation, learning
Centralized	one coordinator (a0)	the same cost model on state polled from every node every 500 ms; no reservations needed
Compute-only	home node of each job	queue wait + nominal execution only; ignores network, data locality, risk and learning
FIFO	one coordinator (a0)	strict arrival order, first node with free capacity, head-of-line blocking; no probing

6. Dashboard GUI

`web/dashboard.html` is one dependency-free page that runs in two modes: live, served by the launcher at `http://localhost:8080/` and refreshed every second, or replay, with recorded runs embedded by `scripts/make_replay.py`.

Panel	Shows
Playback and run selector	run name, policy, scenario, probing mode; a scrubber over the 1 Hz frames; play at 1×, 4× or 12× in replay; follow-live in live mode
KPI strip	jobs done, core and GPU utilisation, throughput, running/queued tasks, data rate, total telemetry rate
Topology	racks, nodes and the core switch; per-node core allocation, queued work, GPUs (× when failed), datasets held, NIC utilisation; rack up/downlink utilisation from the fabric; down or frozen nodes

Panel	Shows
Node inspector	resources, probe interval, decision counters, learned correction factors per workload class, local queue, and the node's distributed view: peer state (fresh / stale / suspect), age, estimated vs true latency and bandwidth
Charts	utilisation (cores, GPUs), throughput vs queued and running tasks, data vs telemetry traffic and latency-estimate error, with fault injections marked
Scheduling decisions	the latest placements with each candidate's cost split into wait, transfer, exec, risk and uncertainty, infeasible candidates with the reason, and the chosen node
Event log	task starts and completions with their wait/transfer/exec split, job completions with JCT, migrations, reschedules, failures and faults
Benchmarks	NEXUS vs centralized vs compute-only vs FIFO per scenario: mean ± sd over seeds for JCT, p95 JCT, makespan, staging time, data moved, utilisation, prediction error, telemetry, reschedules, unfinished jobs

The launcher records every frame it serves, so the replay of a run is exactly what the live dashboard showed.

8. Limitations, future work, build and run

Limitations of this evaluation

- Hardware is emulated. Execution times come from a ground-truth model, so the results measure scheduling and coordination, not kernel performance or memory-bandwidth interference.
- The fabric is a fluid model at 2 MB granularity with FIFO links and switch-buffer-bounded queueing for small packets; it has no TCP congestion control, ECMP or topology beyond one core switch.
- All processes share one 2-vCPU host, so OS scheduling jitter (~0.1 ms) is real and identical across policies; the absolute decision latencies are host-specific.
- The workload is synthetic (Poisson arrivals, five job templates) and runs for 75 s per experiment; three seeds per policy bound but do not eliminate run-to-run variance.

- The centralized baseline polls every 500 ms and is not replicated; a replicated or hierarchical coordinator would narrow the fault-scenario gap.
- Task outputs are assumed durable on the producing node's disk across a process restart.

Future work: run nodes on separate machines with the same binaries (the fabric becomes the real network); learn class weights online instead of fixing them; gang scheduling and co-location constraints for MPI jobs; gossip-based dataset catalogue with replica creation; replay of production traces.

Build and run

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j
(cd build && ctest)                                  # unit tests
./build/nexus_launcher sysinfo                       # host capability
analysis
./build/nexus_launcher run --policy nexus            # live run, dashboard on
http://localhost:8080/
./build/nexus_launcher run --policy centralized --faults
./build/nexus_launcher suite --seeds 3 --hold        # full benchmark (~40
min)
python3 scripts/make_replay.py results results/dashboard_replay.html
python3 scripts/plot_results.py results
```

Requirements: Linux, a C++17 compiler, CMake ≥ 3.16; Python 3 with matplotlib only for the figure script.